

Cuestionario sobre Funciones

Total de puntos 85/100

Las funciones son el corazón de un código limpio y reutilizable. Hemos preparado estas preguntas para repasar cómo declarar, llamar y optimizar funciones en Python de manera efectiva. ¡Tómate tu tiempo y analiza cada fragmento de código!

El correo electrónico del destinatario (**leandrosebastiantrncoso@gmail.com**) se registró al enviar el formulario.

✓ ¿Qué devuelve la función `map(lambda x: x*2, [1, 2, 3])`? * 5/5

- ☐ [2, 4, 6]
- ☒ Un objeto map que al convertirlo en lista produce [2, 4, 6] ✓
- ☐ Un generador que produce 2, 4, 6
- ☐ [1, 2, 3, 2, 4, 6]

✓ ¿Cuál es la forma correcta de escribir una función lambda que reciba dos *5/5 parámetros x e y devuelva su suma?

- ☒ `lambda x, y: x + y` ✓
- ☐ `lambda x y: x + y`
- ☐ `lambda (x, y): x + y`
- ☐ `lambda x,y => x + y`



✓ ¿Qué ventaja ofrece usar funciones como `map()`, `filter()` frente a bucles `for` tradicionales? *5/5

- ☐ Son siempre más rápidas que los bucles.
- ☒ Permiten expresar transformaciones de datos de forma más declarativa y concisa. ✓
- ☐ Eliminan completamente la posibilidad de errores.
- ☐ Obligan a usar list comprehensions.

✓ ¿Cuál es la función de `filter()` en programación funcional? * 5/5

- ☐ Aplica una transformación a cada elemento de un iterable.
- ☒ Selecciona solo aquellos elementos de un iterable que cumplan una condición. ✓
- ☐ Reduce todos los elementos a un único valor acumulado.
- ☐ Ordena los elementos de un iterable basándose en una condición.

✓ ¿Cuál será la salida al ejecutar este código si ejecutamos: * 5/5

```
print(f(1))
print(f(2))
```

```
def f(a, b=[]):
    b.append(a)
    return b
```

- ☐ [1] y [2]
- ☒ [1] y [1,2] ✓
- ☐ [1,2] y [1,2]
- ☐ [1] y [1]



✓ Dado el siguiente fragmento, ¿qué valor imprimirá count si x=5? *

5/5

```
def incremento(x, step=1):  
    count = 0  
    count += x * step  
    return count
```

- ☐ 0
- ☐ 1
- ☒ 5
- ☐ Error, porque count no es accesible



✓ ¿Cuál es el propósito de un parámetro en una función? *

5/5

- ☐ Cambiar el valor de retorno de la función.
- ☐ Almacenar valores en una variable global.
- ☐ Es un valor constante que no puede ser modificado.
- ☒ Recibir datos que se utilizan dentro de la función.



✓ ¿Cuál es la palabra clave utilizada para devolver un valor de una función en Python? *5/5

- ☐ result
- ☒ return
- ☐ value
- ☐ output



✓ ¿Qué se coloca entre paréntesis en la definición de una función? * 5/5

- ☐ Argumentos
- ☒ Parametros
- ☐ Variables
- ☐ Valores



✓ ¿Qué es un "argumento" en el contexto de una función? * 5/5

- ☐ Un valor devuelto por la función.
- ☒ Un valor pasado a la función cuando se llama.
- ☐ Un nombre alternativo para una función.
- ☐ Una función anidada dentro de otra función.



✓ Si defines una función pero nunca la invocas, ¿cuál será la salida? * 5/5

- ☒ No habrá ninguna salida.
- ☐ La función imprimirá un mensaje de error.
- ☐ La función imprimirá el valor predeterminado.
- ☐ La función imprimirá "None".



✓ ¿Cómo se accede al parámetro extra en `def f(x, **kwargs):`? *

5/5

- ☐ `kwargs[0]`
- ☐ `kwargs['x']`
- ☒ `kwargs.get('clave')`
- ☐ `f.extra`



✓ Si una función en Python contiene el código ``return a, b``, ¿qué tipo de objeto recibirá la variable que capture ese resultado?

*5/5

- ☐ Un error de sintaxis, ya que solo se puede devolver un único objeto.
- ☒ Una tupla ('tuple') que empaqueta los valores devueltos.
- ☐ Una lista ('list') que contiene los elementos a y b.
- ☐ Un diccionario con claves numéricas para cada valor.



✗ Al realizar la llamada a una función como ``mi_funcion(valor=10)``, ¿qué tipo de paso de argumentos se está utilizando?

*0/5

- ☐ Argumentos nominales o por nombre.
- ☒ Argumentos por defecto.
- ☐ Argumentos indeterminados.
- ☐ Argumentos posicionales.



Respuesta correcta

- ☒ Argumentos nominales o por nombre.



✓ Dentro de una función, ¿qué estructura de datos representa a los argumentos capturados mediante el uso de `**kwargs`?

*5/5

- ☐ Una tupla.
- ☐ Una lista.
- ☐ Una cadena de texto.
- ☒ Un diccionario.



✓ Si se define una variable `x=10` fuera de una función y otra variable `x=5` dentro de una función sin usar palabras clave especiales, ¿qué sucede al imprimir `x` fuera de la función después de ejecutarla?

*5/5

- ☐ Se imprimirá 5.
- ☐ Ocurrirá un error de nombre ('NameError')
- ☒ Se imprimirá 10.
- ☐ Se imprimirán ambos valores en una tupla.



✗ Al pasar una lista como argumento a una función y modificarla internamente con `.append()`, ¿por qué se refleja el cambio fuera de la función?

*0/5

- ☒ Porque las listas son tipos de datos inmutables por naturaleza.
- ☐ Porque en Python los argumentos se pasan por asignación de objeto.
- ☐ Porque la función devuelve siempre los parámetros modificados.
- ☐ Porque Python siempre crea una copia de seguridad de las listas.



Respuesta correcta

- ☒ Porque en Python los argumentos se pasan por asignación de objeto.



✓ ¿Cuál es la sintaxis correcta para crear una función lambda que calcule el cuadrado de un número n ? *5/5

- ☒ 'lambda n: n**2' ✓
- ☐ 'n=lambda n: n*2'
- ☐ 'def lambda(n): return n**2'
- ☐ 'lambda(n) -> n**2'

✗ En el contexto de la documentación de funciones, ¿qué diferencia a un 'docstring' de un 'type hint'? *0/5

- ☒ El docstring define el nombre de la función, mientras que el type hint define lo que la función retorna. ✗
- ☐ El docstring es un bloque de texto descriptivo para humanos, mientras que el type hint especifica los tipos de datos esperados para herramientas de desarrollo.
- ☐ El docstring es obligatorio para que el código funcione y el type hint es opcional.
- ☐ No hay diferencia; ambos se utilizan exclusivamente para generar documentación automática.

Respuesta correcta

- ☒ El docstring es un bloque de texto descriptivo para humanos, mientras que el type hint especifica los tipos de datos esperados para herramientas de desarrollo.



- ✓ ¿Cuál es la utilidad principal de incluir el bloque `if __name__ == '__main__':` en un archivo de Python? *5/5
- ☒ Permite que el código dentro del bloque se ejecute únicamente cuando el archivo se corre directamente, y no cuando se importa en otro archivo. ✓
 - ☐ Es una medida de seguridad necesaria para que Python pueda compilar las funciones a lenguaje de máquina.
 - ☐ Indica al intérprete de Python que la función `main()` debe ser la primera en cargarse en la memoria RAM.
 - ☐ Sirve para declarar variables globales que serán compartidas automáticamente con todos los demás módulos del proyecto.

El formulario se creó en Universidad de La Punta. - [Comunicarse con el propietario del formulario](#)

¿El formulario parece sospechoso? [Informe](#)

Google Formularios



